Face branch v1 — facial expression recognition (FER) for ThaiSLM

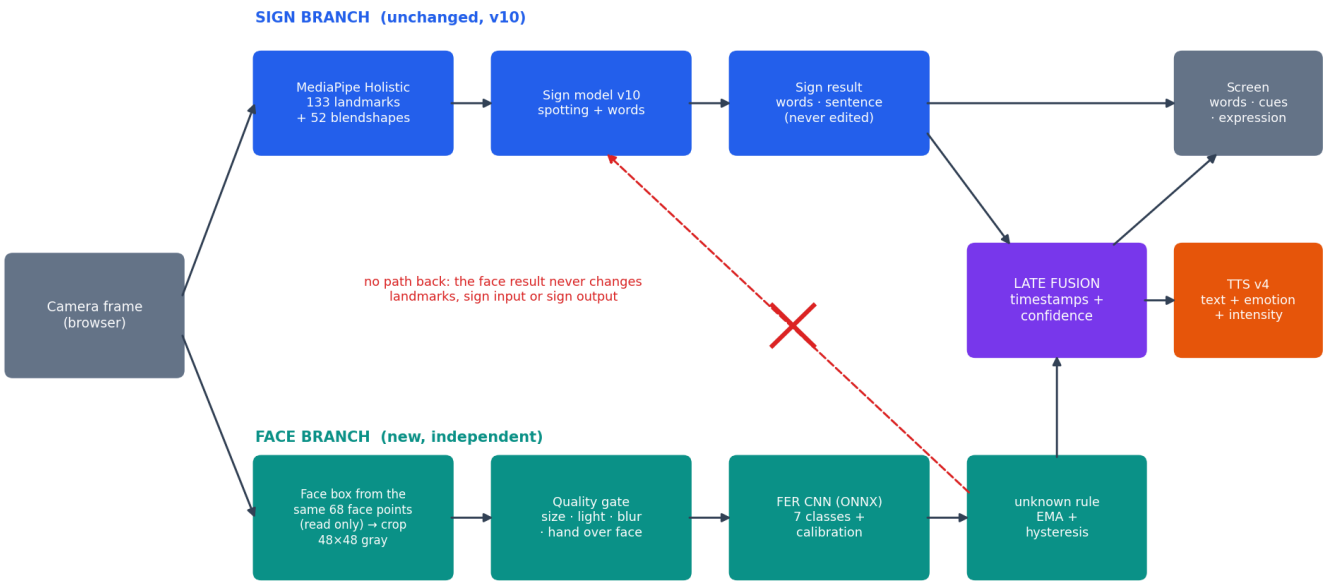
2026-11-02 · model: `models/face-model-efficientnetb0-v1` · code: `modules/face_fer.py`, `Face-Lab/` · demo: `Face-Lab/Face-lab.ipynb`

1. Summary

สรุปสั้น (TH): เพิ่ม "Face branch" ที่อ่านภาพใบหน้าแยกจาก Sign branch โดยสิ้นเชิง (ภาพใบหน้า → โมเดล FER → quality/unknown → EMA + hysteresis → ส่งให้ชั้นเสียง) โมเดลคือ EfficientNet-B0 ที่ fine-tune บน FER2013 ผลบน test 7,178 รูป: accuracy 0.681, Macro-F1 0.663, ECE 0.032, ~5 ms ต่อใบหน้าบน CPU ทดสอบกับระบบจริงผ่าน `docker compose up` แล้ว: ผลของ Sign เหมือนเดิมทุกไบต์ใน 50 การเปรียบเทียบ (รวมกรณี face พัง / ถูกบัง / timestamp เพี้ยน / ปิด face) ส่วนการจูนเสียงตามอารมณ์ (TTS) ยังไม่ได้ทำ — วาง pipeline และสัญญา (contract) ให้เสียบต่อได้แล้ว

What	An *expression category* (angry, disgust, fear, happy, sad, surprise, neutral) or `unknown`, read from the face picture. Never a diagnosis of a feeling.
Model	EfficientNet-B0 (ImageNet-pretrained, fine-tuned on FER2013), 4.0 M parameters, 15 MB ONNX, preprocessing inside the graph
FER2013 test (7,178)	accuracy **0.681**, Macro-F1 **0.663**, Weighted-F1 0.681, UAR 0.670, ECE 0.032, Brier 0.454
Clean test (6,569, no twin in train)	accuracy 0.659, Macro-F1 0.625
Speed	~5 ms / face (ONNX CPU, 2 threads); in the running API: median 107 ms for a whole utterance's ~45 crops, on its own thread
Sign unchanged	10 real clips × 5 conditions through the live Docker API: all sign outputs identical (50/50), sign latency unchanged (median 1,802 ms vs 1,796 ms)
Not done (by design)	tuning the voice to the expression (TTS) — the contract is in place, §8

2. Design



Face crop = pixels of the frame, not landmarks. A face-branch failure, timeout or `unknown` leaves the sign result byte-identical.

Rules that shaped it, and where each is enforced:

- **The face branch is independent.** It reads only a 48×48 gray face crop plus a few numbers (crop size, hand-over-face overlap). It never reads, writes or re-times sign landmarks, sign input or sign output. In the API the sign result is computed first and left untouched; the face result is added under its own key `face_fer` (`sign_modified: false`). Facial *grammar* (yes/no brows, etc.) stays in the sign model; the face branch only reads that flag to avoid mistaking question brows for "surprise".
- **It can fail without hurting anything.** `FaceBranch` never raises: a bad crop, a timeout, a missing model or a covered face gives `unknown` (`ok:false`), and the sign path does not wait for it beyond a small timeout. With the model folder empty the app still starts and translates identically (tested, §6).
- **Late fusion, timestamped.** Browser crops carry timestamps; the server pairs them with sign segments by time. A ±3 s shift of all timestamps changes nothing on the sign side (tested).
- **Stateless server.** The smoother's small state travels with the client, so `--scale sign-api=N` still works.

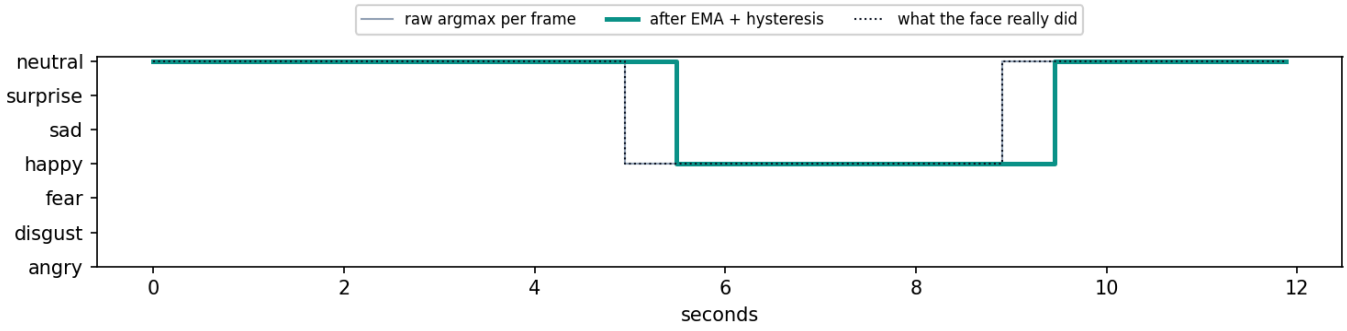


all of this (except T / softmax) is inside the exported ONNX graph · 4.0 M parameters · input name `face`, output `logits`

Crop. MediaPipe's 68 face points give a square box (scale 1.22, shifted 0.16 box-heights up), calibrated so the face fills a FER2013 image the way FER2013 faces do (the 68-point box covers 0.82 of a FER2013 image side, 129 faces measured). The browser sends 48×48 gray crops (base64) at ~5 per second; the server can also crop uploaded videos itself.

Quality gate. Face size, brightness, contrast, sharpness and *hand-over-face overlap* give a quality 0..1. Below 0.25 the frame is `unknown` (hands crossing the face are common in sign language).

Unknown rule. top-1 confidence ≥ 0.48 (chosen on validation: best Macro-F1 while ≥ 80% of frames are answered), margin to the second class ≥ 0.10, quality ≥ 0.25. On FER2013 test this answers 81.9% of images at accuracy 0.745 (Macro-F1 0.717).



Smoothing. EMA of probabilities (τ = 0.35 s) + hysteresis: a new label must lead by 0.12 for 0.3 s (or by 0.8 at once) before it is shown; 1.5 s without a valid frame → `unknown`. One noisy frame cannot flip the label (unit-tested).

Deployment prior. Signers rest on a neutral face, FER2013 does not, so the model's neutral logit gets +0.6 (`prior_shift` in `config.json`). It costs nothing on FER2013 (Macro-F1 0.6633 vs 0.6634) and removes about half the false expressions on real signer faces (§5).

3. Data and training

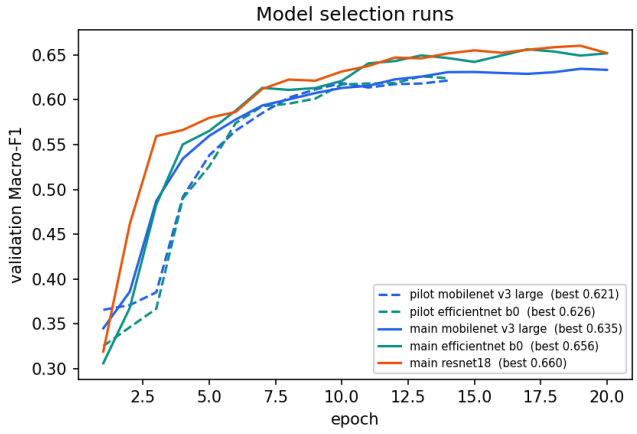
- **FER2013** (`FER2013-Data/`): 48×48 gray, official train/test folders. Class order `[angry, disgust, fear, happy, sad, surprise, neutral]`.
- **Leakage handling.** Test is untouched and never used for selection. 609 test images (568 of them exact copies) also exist in train, exactly or as near-duplicates (dHash) → we report the full test *and* a **clean test** of 6,569 images. Training excludes 108 images: flat images (12) and duplicate images whose copies carry different labels. Validation = 10% of train, stratified and duplicate-safe (25,741 train / 2,860 val).
- **Recipe** (same for every architecture; one local GPU, RTX 2050 4 GB, peak 1.4 GB): AdamW, 1 head-only epoch then all layers, warm-up + cosine, EMA weights, fp16, label smoothing 0.1, sqrt-inverse-frequency class weights (disgust is 11× rarer than happy), GPU-side augmentation (flip, small affine, low-resolution blur, brightness/contrast/gamma, noise, occluding patch), 20 epochs, checkpoint = EMA at best **validation Macro-F1**. Temperature scaling fitted on validation.

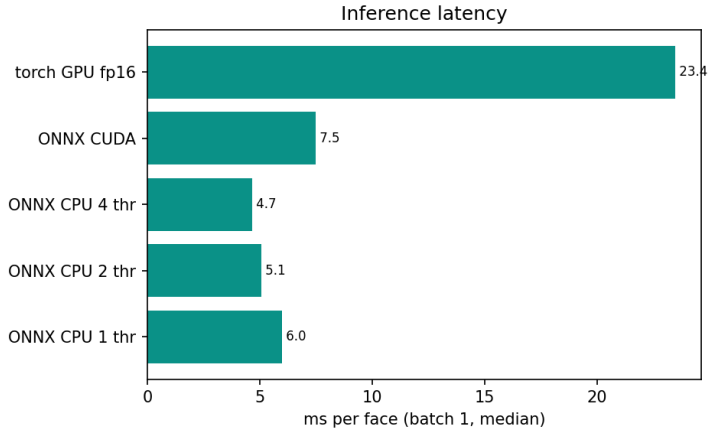
4. Model selection

Three pretrained backbones, same data and recipe:

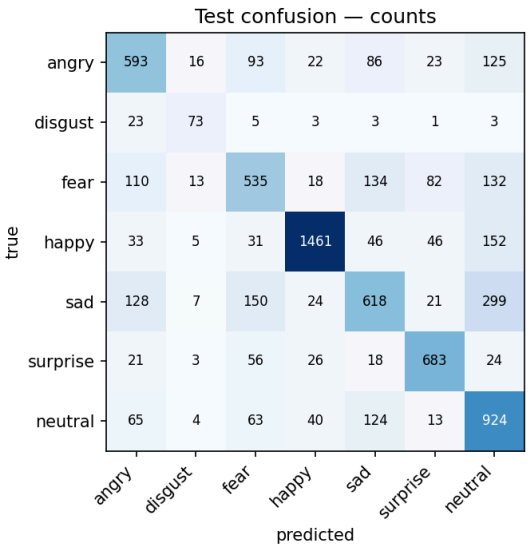
model	params	val Macro-F1	test acc	test Macro-F1	UAR	clean Macro-F1	ECE	CPU ms (2 thr)	train min
MobileNetV3-Large	4.2 M	0.635	0.674	0.663	0.664	0.627	0.035	~2	52
EfficientNet-B0 (chosen)	4.0 M	**0.657**	0.682	0.663	0.671	0.625	**0.030**	~4	78
ResNet-18 (112 px)	11.2 M	0.660	0.696	0.684	0.686	0.649	0.033	~7–11	44

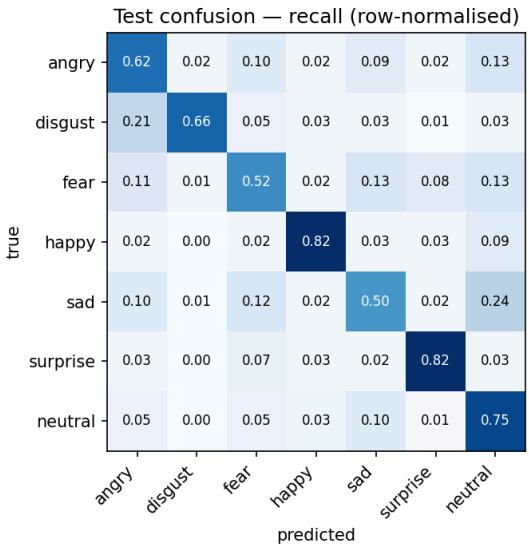
(model-only numbers, temperature scaling, no prior; produced by the notebook, §6 of `Face-lab.ipynb`.)

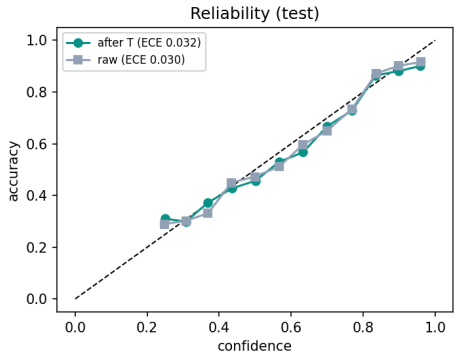


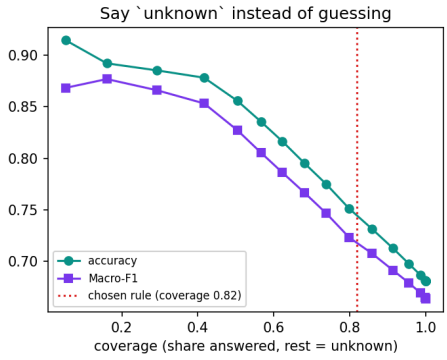


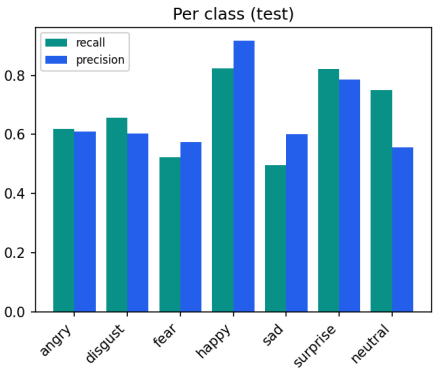
- MobileNetV3 and EfficientNet-B0 are **tied on test** (Macro-F1 0.663 each); B0 was picked by the rule fixed in advance — validation Macro-F1 / UAR / ECE.
- ResNet-18 is the most accurate on FER2013 (+2 points) and trains fastest, **but we did not ship it**: on real signer faces it answers *neutral* for nearly everything — smiles, question brows and a sad line included (neutral is its top choice, 0.33–0.70) — so its very stable output is insensitivity, not accuracy (§5). EfficientNet-B0 recognises those same frames (happy 0.54 / 0.58, surprise 0.59–0.74, sad 0.42).
- Is the model's accuracy too low? In numbers: FER2013 label agreement between humans is ~65%, strong published models reach ~73–76%; we are at 68%. The larger real-world gap is the *domain* (FER2013 stills vs. a signer moving in front of a webcam), which a better FER2013 score does not close.











Weak classes are fear and sad (recall ≈ 0.5, confused with each other and with neutral) — typical for FER2013. Calibration is good (ECE 0.032); accuracy of answered frames rises from 0.68 to 0.75 when `unknown` is allowed.

5. Real signer faces (18 clips: Boom, Jack, Pitty)

There are **no expression labels** for the clips. We sampled a face crop every 0.5 s (356 crops), looked at them, and built a **small, hand-judged** set: 3 clear smiles, ambiguous frames left out (55), frames with the face covered by a hand skipped (41, the app answers `unknown`), every other frame counted as neutral (257). This measures *stability / over-reacting*, not accuracy. `Face-Lab/real_check.py` reproduces it.

On frames judged neutral, the label the app would show (after unknown rule + smoother):

	shown neutral	shown a wrong expression	shown unknown	smiles shown happy
EfficientNet-B0, no prior	64.6%	12.5%	23.0%	3/3
EfficientNet-B0 + neutral prior 0.6 (shipped)	**82.1%**	**6.6%**	11.3%	2/3
ResNet-18 (not shipped)	94.6–98.1%	1.6–2.3%	0.4–3.1%	0–1/3

Per signer (shipped): Jack 96% neutral / 0% wrong · Boom 83% / 9% · Pitty 68% / 10% (Pitty's resting face seems to have raised brows and wide eyes, which the model reads as fear — the prior removed most of it, not all).

Frames where something *is* happening (model only, no prior): smiles → happy 0.54, 0.58, 0.30 (vs neutral 0.27); question brows → surprise 0.59, 0.67, 0.74; sad line → sad 0.42, 0.32. Honest misses: a head-thrown-back sign (Jack, "เรียน") reads *disgust 0.89*; a few blinks read *sad*. Frames with a hand over the face are correctly `unknown`.

Limits: the judged set is tiny and made by us; there are only 3 clear smiles and the clips contain few real expressions. A proper real-world evaluation needs expression labels on sign videos (next steps, §8).

6. Real-system test (docker compose up)

All services of the compose file were started (gateway, web, sign-api, tts on GPU, reverse-api, reverse-speech) and became healthy; `/api/v1/health` reports `face: face-model-efficientnetb0-v1`, `/api/v1/face/info` returns the config. Memory: sign-api 0.79 GB in total with the face model loaded.

test	result
Non-interference (`Face-Lab/non_interference.py`, 10 real clips: Jack and Pitty; 5 conditions each): baseline without face data vs real crops, garbage crops (noise / black / wrong size / bad base64), timestamps ±3 s, face covered	every sign output (segments incl. scores and embeddings, sentence, language, face cues, duration) **identical, 50/50**; only `timings_ms`, `face_fer` and the voice choice may differ
Sign latency with vs without face crops	median 1,802 ms vs 1,796 ms; face branch itself median 107 ms per utterance, run on its own thread
FER result under conditions	real crops → neutral (valid frames 27–93%); garbage crops → unknown; covered face → unknown
Face disabled (model folder empty, `FACE_FER_DIR`)	app healthy, `face: null`, `/v1/face` answers `{"ok":false, "error":"face branch unavailable"}`, sign output identical
Video upload (server crops faces itself)	`face_fer` returned, `sign_modified:false`, sentence unchanged
UI, upload	result card shows the sentence and the sign chips; no errors from our code
UI, live mode	the built-in browser has no camera, so a recorded clip was fed through a patched `getUserMedia`: in-browser MediaPipe, face crops posted to `/api/v1/face`, overlay label on the camera, result card line "สีหน้า (จากภาพ): เป็นกลาง 65%". The only console errors are MediaPipe's WebGL-delegate fallback messages (not ours).
`/api/v1/face` with FER2013 faces (4 emotions)	smoothed label correct for happy, surprise, neutral and sad
Notebook webcam cell	ran on the real webcam at 29 fps (no face was detected in those frames, so the expression log is empty)
Unit tests	`tests/test_face_branch.py`: 8 tests (crop geometry, quality gate, hand overlap, smoother, unknown after loss + state round trip, voice rules, never-raises, determinism) pass

7. Known limits

- FER2013 is low-resolution, noisy-labelled stills; the model had **no sign-language video** in training. Real signers see more `unknown` (raw ≈ 30% on neutral frames) and occasional wrong expressions (6.6% of neutral frames).
- Mouthing, blinking, head movement and sign grammar (raised brows, open mouth) all look like FER classes. The smoother and the unknown rule reduce, not remove, this.
- "disgust" and "fear" are the least reliable labels in practice.
- The model describes an expression, not an emotion.

8. Plugging in the TTS later (contract)

The speech service takes **text + emotion + intensity** (`POST /v1/speak {text, voice, emotion, intensity}`; emotion = the 7 FER2013 names, intensity 0..1; verified through the gateway: audio differs per emotion). The API already hands the three values over:

- `face_fer.summary` — utterance expression, probabilities, `valid_frac`, quality, stability (plus the per-frame `track` and `per_segment` results);

- `speak_text`, `speak_emotion`, `speak_intensity` — what the web sends to the voice, decided by `speech_affect()` in `modules/face_fer.py`.

`speech_affect()` is deliberately conservative (it voices only a confident non-neutral emotion: 0.5 FER + 0.5 v9 blendshape affect, ≥ 0.5, subtle intensity 0.25–0.75, surprise suppressed on yes/no questions), so on today's mostly-neutral clips it voices nothing (10/10 clips: `speak_emotion = null`). Once the TTS is tuned, only this one function (or its thresholds) needs to change — nothing else in the face branch or the web. The Face-Lab never touched the TTS model.

9. Next steps

1. Label expressions on a few sign videos (smile, question brows, sad, neutral) → a real validation set; fine-tune / calibrate on it (the largest expected gain).
2. Tune the voice for the expression, then set the thresholds in `speech_affect()`.
3. Per-class thresholds (disgust, fear) once the real set exists; optional flip averaging (+0.7 points on FER2013, but it lost smiles in our tiny real set).

10. Reproduce

```
python Face-Lab/train.py --arch efficientnet_b0 --res 96 --epochs 20 --head-epochs 1 --unfreeze 0.0 --tag
main_efficientnet_b0
python Face-Lab/finalize.py --run runs/main_efficientnet_b0 --name v1      # bundle -> models/face-model-
efficientnetb0-v1
python Face-Lab/make_figures.py all
python Face-Lab/real_check.py                                               # real signer faces
docker compose up -d && python Face-Lab/non_interference.py --clips <video> ...
python tests/test_face_branch.py -v
```